

eBPF: Kernel Plugins for Security, Observability, and More

Let's see how eBPF works and get hands-on with some sample programs.

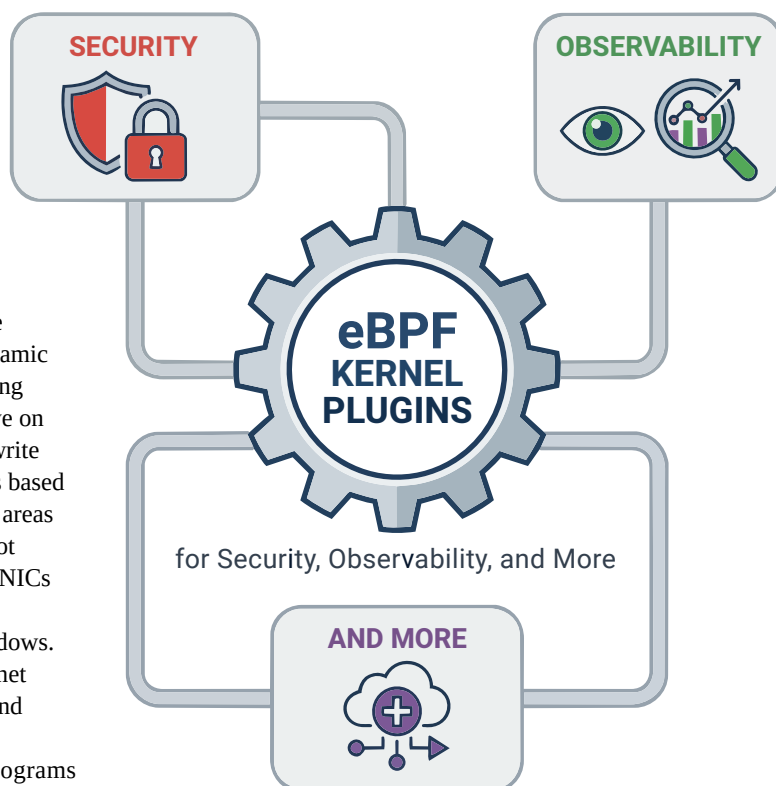
eBPF is surprisingly popular for a kernel-level technology. Forget the books and tutorials – there is a foundation, an ACM SIGCOMM workshop, and even a documentary about it! Why so? Marketed as ‘JavaScript for the kernel’, eBPF lets us write high-performance dynamic plugins for the Linux kernel, with use cases ranging from tracing to custom schedulers. Keeping an eye on eBPF is a smart move even if you don't want to write eBPF programs manually – tools and frameworks based on this technology are making a revolution in the areas of observability and security. This revolution is not just limited to Linux-based hosts; there are SmartNICs (hardware) that support eBPF, and Microsoft is actively working on adding eBPF support to Windows. According to ebpf.io, eBPF is used by large Internet companies including Google, Meta, Cloudflare, and Netflix for one purpose or another.

What makes eBPF powerful is that eBPF programs get to run in kernel space, where it can hook to so many critical points and intercept operations to tap and manipulate data. You can do a lot of things at these levels that you cannot do in user space (like participating in process scheduling). You get to do so many things sooner as well (like checking and dropping a packet before it even enters the kernel network stack).

So what makes eBPF so special, especially compared to existing solutions like kernel modules which also help us extend the kernel? Well, kernel modules can be far more capable than eBPF programs – you are essentially developing a chunk of the kernel. But eBPF programs do not have the same level of freedom. They have many restrictions enforced by the kernel. While this sounds disappointing, these restrictions actually make extending the kernel a fearless activity – there is no way we can crash the kernel with a buggy eBPF program unless there is a bug in the kernel itself.

Powered by eBPF

There are celebrated projects like Cilium that bring massive performance and security benefits to the cloud with the help



of eBPF. Cilium is a high-performance Container Networking Interface for Kubernetes. It replaces the traditional kube-proxy and iptables-based workflow, which has significant overhead compared to eBPF-based load balancing and routing done at the lowest level possible. However, let's focus on some other tools that are both powerful enough to be used on real servers and simple enough to try on our personal laptops.

Since eBPF started out as an extended version of the classic ‘Berkeley Packet Filter’, it has found its way into some existing firewall solutions, and newer ones are being built based on it. One noteworthy solution is `bpfilter`, which translates network filtering rules to eBPF programs. We do not have to manually load these programs; `bpfilter` will take care of that for us (in fact, `bpfilter` originated as a drop-in replacement for `iptables`).

`bpfilter` comes with a command-line utility called `bfcli`, which is used to install and manipulate filtering chains. Here is an example that drops all ICMP packets, essentially blocking ping: